



FOR SOFTWARE ENGINEERS

All Green. Still Wrong.

Stop reading the code? Uncle Bob's rule, tested on a real agent

A gate only catches what its tests, rules or analysis know how to recognise.

Robert Martin wrote in April 2026 that he no longer reviews code written by agents; he measures coverage, dependency structure, complexity, module size and mutation testing instead. We built those gates, gave a coding agent a real refund feature from a frozen spec, and attacked the green build.

Every result below was measured, and one script in the repository reruns it.

Versions and advisories change. Everything on this sheet was true on 28 September 2026: Spring Boot 4.1.1, Java 25, JaCoCo 0.8.15, PIT 1.30.0, ArchUnit 1.5.1, Semgrep 1.175.0, osv-scanner 2.5.1. The method is the durable part: know what each gate can recognise.

Measured

- One agent run on one feature, kept as it came out
- Six calibration changes, each aimed at one gate
- Five attacks on the green build, each also bending the test that guarded it
- A fault injection for the concern found by reading

Not measured here

- How reliably human reviewers would find the same concerns (no blind review was run)
- Other agents, other features, other codebases
- The paid editions of the scanners

Fictional payments data. Five passes of a concurrency test are evidence, not a proof.

01 The findings, in one table

AGENT, first run	7.4 min, 614 lines; all 7 visible gates; 12 of 12 hidden tests
CALIBRATION	coverage, complexity, architecture and dependency gates each caught theirs
NO-ASSERTION TEST	passed PIT: build 82% vs 80% bar, that class 0 of 2
SQL INJECTION	flagged in one method, missed one call away (Semgrep CE)
READING	provider called inside the transaction: one refund, two payouts
RECEIPT OWNERSHIP	visible gates pass; acceptance tests catch it
ONE QUERY PER REFUND	visible gates pass; query budget catches it
DOUBLE REFUND RACE	visible gates pass; acceptance tests catch it
CARD WINDOW 14 TO 30	tests bent to agree; only the spec tests catch it
RETIRED LEGACY PATH	all nine gates pass

Four of five attacks were stopped by independently written checks.

02 The gates

Seven gates the agent could see and run, and two it never saw. A gate prints PASS or FAIL; the thresholds live in the build.

```
scripts/gates.sh: nine gates, one line each
```

```
gate tests          ./mvnw -q -B -Djacoco.haltOnFailure=false -Dtest='!ArchitectureTest' test
gate coverage       ./mvnw -q -B jacoco:check@check
gate mutation       ./mvnw -q -B pitest:mutationCoverage
gate complexity     ./mvnw -q -B pmd:check
gate architecture   ./mvnw -q -B -Djacoco.skip=true -Dtest=ArchitectureTest test
gate security       semgrep scan --config p/java --error --quiet --metrics=off src/main
gate dependencies   osv-scanner scan source --recursive .

if [ $HIDDEN = 1 ]; then
  cp -r "$ROOT/hidden/dev/example/payments/acceptance" src/test/java/dev/example/payments/
  gate acceptance   ./mvnw -q -B -Djacoco.skip=true -Dtest=RefundAcceptanceTest -
  DexcludedGroups=query-budget test
  gate query-budget ./mvnw -q -B -Djacoco.skip=true -Dtest=RefundAcceptanceTest -Dgroups=query-
  budget test
fi
```

```
service/pom.xml: the numbers a change must clear
```

```
<gate.line-coverage>0.90</gate.line-coverage>
<gate.branch-coverage>0.85</gate.branch-coverage>
<gate.mutation-score>80</gate.mutation-score>
```

```
ArchitectureTest.java
```

```
static final ArchRule controllersDoNotReachRepositories = noClasses()
    .that().areAnnotatedWith(RestController.class)
    .should().dependOnClassesThat().areAnnotatedWith(Repository.class);
```

On the empty project, before any code, the dependency gate failed: Spring Boot 4.1.1 ships Tomcat 11.0.24, and osv-scanner reported three findings it rated critical.

Measured on 2026-09-28. Reproduce it with `scripts/gates.sh` and `experiments/variants.py`.

03 What reading found

Every gate passed the agent's code. Reading it showed the payment provider is called while the database transaction is open. The fault injection makes the first update after the payout fail: the money moves, the refund stays PENDING, and the retry pays again.

RefundService.java, as the agent wrote it

```
@Transactional
public Refund approve(long refundId) {
    Refund refund = refunds.findByIdForUpdate(refundId)
        .orElseThrow(() -> new NotFoundException("No refund " + refundId));
    if (refund.isRefunded()) {
        return refund;
    }
    provider.refund(refund.paymentId(), refund.amountCents());
    return refunds.markRefunded(refundId);
}
```

CommitFailsAfterPayoutTest.java: the double payout, reproduced

```
assertThatThrownBy(() -> refunds.approve(refund));
assertThat(Money.PAYOUTS.get()).as("paid out on the first approval").isEqualTo(1);
assertThat(db.sql("SELECT status FROM refunds WHERE id = :id").param("id", refund)
    .query(String.class).single()).as("but the refund still says").isEqualTo("PENDING");

refunds.approve(refund);
assertThat(Money.PAYOUTS.get()).as("payouts after a retry").isEqualTo(2);
```

One refund, two payouts, with the commit made to fail once. The spec never said what happens when a commit fails after money moves.

experiments/agent-run: the prompt, the full transcript and the agent's change, first run kept.

04 What a gate can see

The same injection, in the method that receives the request, is flagged. Moved one call away it is not: Semgrep Community Edition "can only analyze interactions within a single function" (docs.semgrep.dev, read 28 September 2026).

Split.java: one call away, Semgrep CE finds nothing

```
@GetMapping("/a")
public List<Map<String, Object>> list(@RequestParam String sort) {
    return sorted(sort);
}

private List<Map<String, Object>> sorted(String sort) {
    return client.sql("SELECT * FROM payments ORDER BY " + sort).query().listOfRows();
}
```

A mutation threshold is an average: one class with a zero percent mutation score hid inside an 82 percent build.

Measured on 2026-09-28. Reproduce it with `scripts/gates.sh` and `experiments/variants.py`.

05 The questions this answers

Can you stop reviewing agent-written code if the gates are strong enough?

What does mutation testing prove, and what does its threshold hide?

Why can a green build encode the wrong business rule?

Why should a payout never happen inside the transaction that records it?

What can a single-function static analysis not follow?

Before you stop reading

1. Write acceptance tests from the spec, separately from the implementation
2. Read the per-class mutation report, not just the build total
3. Know which scanner edition you run and what it cannot follow
4. Keep money movement out of the transaction that records it
5. Budget queries per request, so an N+1 fails a build
6. Write down the context: retired paths, owners, rules

Everything else

-  **Site**
<https://code-with-sam-dev.github.io>
-  **YouTube**
<https://www.youtube.com/@CodewithSam-Dev>
-  **LinkedIn**
<https://www.linkedin.com/company/code-with-sam-dev>
-  **x**
<https://x.com/CodeWithSamDev>
-  **TikTok**
<https://www.tiktok.com/@codewithsamdev>
-  **Instagram**
<https://www.instagram.com/codewithsamdev>
-  **Facebook**
<https://www.facebook.com/codewithsamdev>
-  **GitHub**
<https://github.com/code-with-sam-dev>
-  **Buy me a coffee**
<https://buymeacoffee.com/codewithsam>
-  **Buy me sadza**
<https://buymesadza.com/codewithsam>
- The spec, gates, agent transcript and every attack**
<https://github.com/code-with-sam-dev/stop-reading-the-code>
- Semgrep: cross-function analysis**
<https://docs.semgrep.dev/semgrep-code/semgrep-pro-engine-intro>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Semgrep is a trademark of Semgrep, Inc. This is an independent, unofficial build produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-09-28 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.